

Ereditarieta, interfacce e trait

Ereditarieta, interfacce e trait

Come riusare e organizzare comportamento tra classi PHP.

Prima la soluzione semplice

Ereditarieta, interfacce e trait sono strumenti utili, ma non servono in ogni programma. Prima prova con classi piccole e metodi chiari. Usa questi strumenti quando risolvono un problema reale.

Ereditarieta con extends

L'ereditarieta permette a una classe di partire da un'altra classe.

```
<?php
class Utente {
    public function saluta(): string {
        return "Ciao";
    }
}

class Admin extends Utente {
    public function pannello(): string {
        return "Area amministrazione";
    }
}

$admin = new Admin();
echo $admin->saluta();
```

`Admin` eredita il metodo `saluta` da `Utente` .

Interfacce con implements

Un'interfaccia descrive quali metodi una classe deve avere.

```
<?php
interface Notificabile {
    public function invia(string $messaggio): void;
}

class Email implements Notificabile {
    public function invia(string $messaggio): void {
        echo "Email: $messaggio";
    }
}
```

L'interfaccia non spiega come fare il lavoro. Stabilisce il contratto.

Trait

Un trait permette di riusare metodi in più classi.

```
<?php
trait LoggaAzioni {
    public function log(string $messaggio): void {
        echo "Log: $messaggio";
    }
}

class Ordine {
    use LoggaAzioni;
}
```

Quando usarli

Usa `extends` quando una classe è davvero una versione più specifica di un'altra. Usa un'interfaccia quando classi diverse devono rispettare lo stesso contratto. Usa un trait per condividere piccoli comportamenti, senza costruire una gerarchia complicata.

Un esempio di interfaccia con piu classi

Il valore di un'interfaccia si vede quando classi diverse possono essere usate nello stesso modo.

```
<?php
interface Pagabile {
    public function paga(float $importo): void;
}

class CartaCredito implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con carta: $importo";
    }
}

class Bonifico implements Pagabile {
    public function paga(float $importo): void {
        echo "Pagamento con bonifico: $importo";
    }
}
```

Entrambe le classi promettono di avere un metodo `paga` .

Quando evitare l'ereditarieta

Non usare `extends` solo per riusare due righe di codice. L'ereditarieta crea un legame forte tra classi. Se il legame non rappresenta davvero un rapporto "e un tipo di", una funzione, una composizione o un trait piccolo possono essere piu semplici.

Riepilogo

Questi strumenti servono a organizzare progetti piu grandi. Se una pagina con classi semplici e metodi chiari risolve il problema, resta su quella strada. La semplicita e una scelta tecnica, non una mancanza.